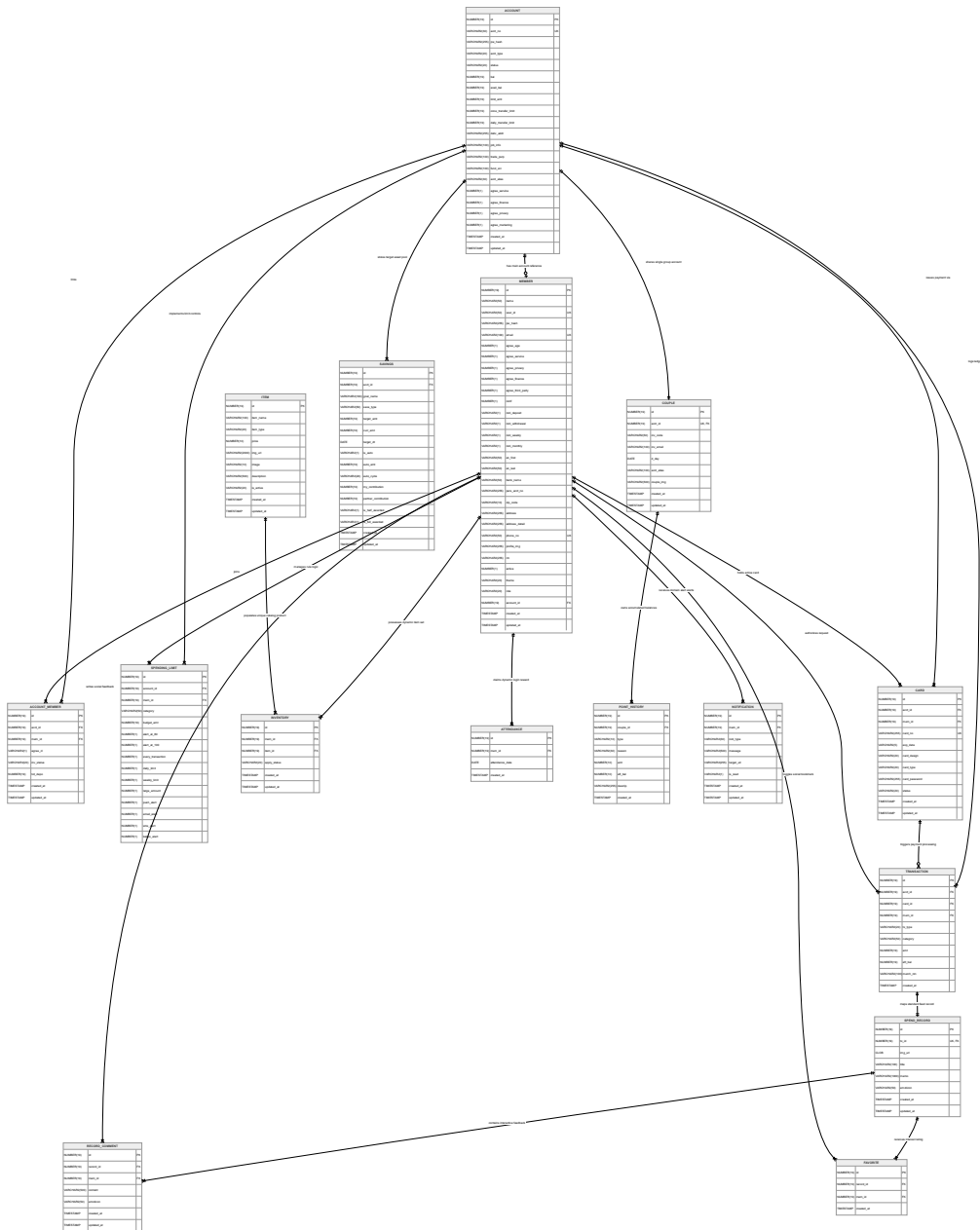


1. ERD

🕒 생성일	2026년 4월 4일 오후 10:26
🏷️ 태그	

📊 데이터베이스 ERD 및 설계 의도 (Data Architecture & Design Intent)



본 문서는 MODAM 프로젝트의 핵심 데이터 흐름과 도메인 간의 관계를 시각화한 ERD(Entity Relationship Diagram)와, 이를 도출하기 위한 백엔드 데이터베이스 설계 주안점을 명세합니다.

(세부 테이블 컬럼 및 데이터 타입 명세는 **[DB 설계서]**를 참조해 주세요.)

1. ERD 다이어그램 (Entity Relationship Diagram)

2. 핵심 데이터 모델링 및 설계 의도 💡

본 프로젝트의 DB 아키텍처는 단순한 데이터 저장을 넘어, 금융 서비스의 **정합성**, **동시성 이슈 방어**, **시스템 보안**을 데이터베이스 계층(Entity Level)에서 물리적으로 보장하도록 설계되었습니다.

🛡️ 1. 금융 거래 정합성 및 무결성 보장

- **ACCOUNT (1) ↔ TRANSACTION (N)**
- 모든 입출금 및 카드 결제 이력(**TRANSACTION**)은 원천 계좌(**ACCOUNT**)와 1:N 관계로 엄격하게 바인딩됩니다.
- 단순히 현재 잔액만 업데이트하는 것이 아니라, 트랜잭션 테이블 내에 **balance_after (거래 후 잔액)** 컬럼을 역정규화하여 스냅샷으로 보관합니다. 이를 통해 향후 데이터 정합성 훼손 시 장애 지점을 역추적하고 복구할 수 있는 원장(Ledger) 시스템을 구현했습니다.

🔗 2. 1:1 매핑을 통한 영수증(소비 기록) 무결성 강제

- **TRANSACTION (1) ↔ SPEND_RECORD (1)**
- 결제가 발생한 금융 데이터(**TRANSACTION**)와 커플이 남기는 사진/메모 추억(**SPEND_RECORD**)은 논리적으로 1:1 관계입니다.
- 이를 보장하기 위해 **SPEND_RECORD** 테이블의 **tx_id** 외래키에 **물리적 유니크(Unique) 제약 조건**을 부여했습니다. 하나의 결제 건에 여러 개의 추억 영수증이 중복 생성되는 논리적 오류를 DB 단에서 원천 차단했습니다.

⚡ 3. 동시성 이슈 대응 및 포인트 오염 방지 (따닥 방어)

- **MEMBER (1) ↔ INVENTORY (N) ↔ ITEM (1)**
- 사용자가 상점에서 아이템 구매 버튼을 비정상적으로 연타하여 동일한 아이템이 중복 결제되고 포인트가 이중 차감되는 동시성 이슈(Concurrency Issue)를 고려했습니다.
- 애플리케이션 레벨의 검증에 더해, **INVENTORY** 테이블에 ****uk_mem_item (mem_id, item_id 복합 유니크 제약)****을 설정하여 물리적인 데이터 오염을 완벽하게 방어했습니다.

🔒 4. 양방향/단방향 암호화를 통한 보안 컴플라이언스

- 계좌 비밀번호 및 로그인 비밀번호(**pw_hash**)는 **BCrypt 단방향 암호화**를 적용하여 복호화가 불가능하도록 처리했습니다.
- 실물/모바일 카드 번호(**card_num_enc**) 등 역추적이 필요한 민감한 금융 데이터는 **AES-256 양방향 암호화 알고리즘**을 거쳐 DB에 적재되도록 설계하여 정보 유출에 대비했습니다.

🎮 5. 게이미피케이션 어뷰징 방지 및 Audit (감사)

- 저축 목표(**SAVINGS**) 테이블에 **is_half_awarded**, **is_full_awarded** 플래그를 두어 목표 달성 이벤트에 대한 포인트 중복 지급을 방지했습니다.
- 모든 엔티티에는 Spring Data JPA의 ****@EntityListeners(AuditingEntityListener.class)****를 적용하여, **created_at** (생성일)과 **updated_at** (수정일)이 애플리케이션 로직 누락 없이 시스템 시간에 의해 자동으로 로깅되도록 구현했습니다.